

DEV2 – JAVL – Laboratoire Java**TD 10 – Polymorphisme****Résumé**

Dans ce TD, nous allons aborder la notion de polymorphisme, ceci nous permettra de traiter certains objets de classes différentes comme des objets de la même classe. Pour faire ceci, il faudra que les objets héritent d'une même classe parent.

Table des matières

1 Le casting	1
2 L'animalerie	2
3 Les contacts professionnels	3

1 Le casting

Avant d'aborder la notion de polymorphisme, nous allons introduire la notion de conversion forcée, qui est essentiel pour le polymorphisme. Le conversion forcée est le processus permettant de convertir une variable d'un type de données à un autre. En Java, la conversion peut être divisé en deux catégories principales : la "conversion implicite" et la "conversion explicite", aussi appelée **casting**.

Exécutez le code suivant :

```
int int_a = 5;  
double double_a = int_a;  
System.out.println(double_a);  
double double_b = 5;  
int int_b = double_b;  
System.out.println(double_b);
```

Si vous lancez ce code, vous allez remarquer qu'une erreur apparaît à la ligne 5. Remplacez cette ligne par : `int int_b = (int) double_b;` ; l'erreur disparaît.

La ligne `double double_a = int_a;` est ce que l'on appelle une conversion implicite, puisque l'on attribue à une variable `double` la valeur d'un type de variable différent (`int`). Ici, Java comprend ce que l'on essaie de faire et va procéder à une conversion automatique. La ligne `int int_b = (int) double_b;` représente une conversion explicite. Cette fois-ci, on attribue une valeur `double` à un type de variable entière. Le problème est qu'une variable double est plus précise qu'une variable entière (en effet, celle-ci peut avoir des décimales), la conversion pourrait nous faire perdre de l'information (ce qu'il y aurait après la virgule). Pour éviter cette perte d'information Java ne fait pas de conversion automatique dans ce sens-là, d'où l'erreur que l'on avait précédemment. Il faut alors

lui préciser que l'on veut uniquement la partie entière de notre double grâce à la ligne `int int_b = (int) double_b;` c'est ce que l'on appelle la conversion explicite (ou casting).

Casting

Le cast est le fait de forcer le compilateur à considérer une variable comme étant d'un type qui n'est pas le type déclaré ou le type réel de la variable.

Exercice 1 Casting chien et animal

- Créez un `Dog` nommé Rex et pesant 10 kg. Castez Rex dans une variable `Animal`. Quel type de conversion devez-vous utiliser ? Pourquoi ?
- Créez un `Animal` nommé Duke et pesant 30 kg. Castez Duke dans une variable `Dog`. Quel type de conversion devez-vous utiliser ? Pourquoi ?

2 L'animalerie

Pour aborder la notion de polymorphisme nous allons d'abord commencer par créer une nouvelle classe nommée `PetStore`.

Exercice 2 Animalerie

Créez une classe `PetStore`, qui possède un attribut `animals` qui est une liste d'`Animal`.

- créez un constructeur, il prend en paramètre une liste d'`Animal` et il attribue cette liste à `animals` ;
- faites une surcharge du constructeur de cette classe, en créant un nouveau qui ne prend pas de paramètre et initialise une liste vide.
- créez un accesseur pour l'attribut `animals` ;
- créez une méthode `addAnimal` qui ajoute un `Animal` passé en paramètre à l'attribut `animals` ;
- créez une méthode `removeAnimal` qui retire un `Animal` passé en paramètre de l'attribut `animals`.
- redéfinissez le `toString` pour qu'il affiche l'ensemble des animaux de l'attribut `animals`.

Si, comme mentionné au précédent TD, vous avez modifié le `toString` de votre classe `Animal`, vous devriez voir s'afficher le nom et le poids des animaux mais vous n'avez pas l'information si il s'agit d'un chat ou d'un chien.

Exercice 3 Chien ou chat

Utilisez un `@Override` pour la méthode de `toString` dans la classe `Dog` de façon à obtenir un affichage comme ceci : `Chien Medor : poids = 60kg`. Faites de même pour la classe `Cat` (en remplaçant évidemment chien par chat).

Nous allons commencer à utiliser le polymorphisme. Le polymorphisme en Java se réfère à la capacité d'un objet d'une classe à être utilisé comme un objet de son type de base (classe parent). Le polymorphisme permet une flexibilité dans la conception des classes en permettant l'utilisation générique du code tout en conservant la capacité d'ajouter des comportements spécifiques aux sous-classes. Cela favorise la réutilisation du code et une meilleure gestion de la complexité.

Polymorphisme

Le polymorphisme est un concept qui permet de traiter des objets de différents types comme des objets d'une superclasse commune.

Exercice 4

L'animal chien

Dans une classe `Main`, tapez le code suivant :

```
Animal medor = new Dog("Medor", 60)
System.out.println(medor);
medor.bark()
```

Dans ce cas `medor` est bien de type `Animal` pourtant il est défini comme un `Dog`, nous pouvons faire cela, car `Dog` hérite de `Animal`.

Pourrions-nous faire le contraire ?

Pourquoi ?

Vous obtenez une erreur, à la troisième ligne, pourquoi ?

Supprimez la troisième ligne, remarquez ce que vous donne le `System.out.println(medor)` correspond il au `toString` de `Animal` ou de `Dog` ? Pourquoi ?

Vous avez ici un exemple de polymorphisme où un objet (`medor`) va être traité comme un `Animal`, mais est en réalité un `Dog`.

Exercice 5

Affichez l'animalerie

De façon similaire à l'exercice précédent, définissez 6 objets `Animal`, trois d'entre eux sont des chiens et trois sont des chats. Définissez une variable `petstore` de type `PetStore`, ajoutez ces animaux à l'animalerie.

Affichez l'animalerie grâce à `System.out.println(petstore)`. Remarquez encore une fois, que comme à l'exercice précédent, l'affichage nous précise s'il s'agit d'un chat ou d'un chien, cela illustre encore le polymorphisme.

Exercice 6

Faites du bruit

Créez une méthode dans la classe `PetStore` nommée `makeNoise` qui va faire aboyer ou miauler tous les animaux de l'animalerie, c'est-à-dire les éléments de l'attribut `animals`. Essayez ces méthodes dans le `main`.

Astuce : utilisez la méthode `sound` plutôt que les méthodes `bark()` et `meow()`.

Exercice 7

Donnez tous les chiens

Créez une méthode dans la classe `PetStore` nommée `allDog` qui retourne une liste de tous les chiens dans l'animalerie. Faites de même pour les chats avec une méthode nommée `allCat`. Essayez ces méthodes dans le `main`.

Astuce : pour résoudre cet exercice, vous pouvez utiliser l'opérateur `instanceof`, celui-ci permet de savoir si un objet appartient à une classe ou non. Par exemple, `medor instanceof Dog` rend `true` si la variable `medor` est bien une instance de la classe `Dog`, il rend `false` dans le cas contraire.

3 Les contacts professionnels

Exercice 8

Contacts professionnels

Reprenez vos classes `Contact` et `PhoneBook`. Créez une nouvelle classe `BusinessContact` qui hérite de `Contact`. Cette nouvelle classe possède trois attributs supplémentaires, le

nom de l'entreprise (`String companyName`), l'adresse de l'entreprise (`String companyAddress`) et le numéro de téléphone de l'entreprise (`int companyPhone`). Définissez un constructeur ainsi que les accesseurs de cette classe.

Exercice 9

Modifiez l'annuaire

Modifiez la classe `PhoneBook` :

- ▷ modifiez la méthode `display()` pour qu'elle affiche également les informations de l'entreprise, si le contact est un contact professionnel ;
- ▷ créez une surcharge de la méthode `update` pour que cette version prenne également en compte les informations de l'entreprise ;
- ▷ créez une méthode `becomeProfessional` qui prend un argument un `Contact` (le contact à changer dans la liste), deux `String` (le nom et l'adresse de l'entreprise) et un `int` (le numéro de téléphone de l'entreprise). Cette méthode transforme le `Contact` donné en paramètre en `BusinessContact`. Si le contact n'existe pas, la méthode lance une exception. Pensez à redéfinir le `equals` et le `hashCode` de `Contact` si nécessaire.

Exercice 10

Nombre de contacts privés et professionnels

Écrivez les méthodes suivantes :

- ▷ une méthode `nPrivateContact` qui renvoie le nombre de contacts privés dans l'annuaire (un contact privé est un contact qui n'est pas un `BusinessContact`) ;
- ▷ une méthode `nCompanyContact` qui renvoie le nombre de contacts professionnels dans l'annuaire, c'est-à-dire le nombre de `BusinessContact`.

Exercice 11

Contacts dans une entreprise

Créez une méthode `inCompany` qui prend un paramètre un `String` représentant le nom d'une entreprise. Cette méthode renvoie la liste de tous les contacts professionnels appartenant à cette entreprise.